PH142 Review: Week 1

PH142 GSI Team
Summer 2026

Announcements

- **Quiz04:** due 7/9 at 11:59pm
- **Quiz05:** due 7/10 at 11:59pm
    - Daily quizzes are available all day (24h)
    - Once opened, you have 30 minutes to complete the quiz

Learning Objectives

- Key technologies & Resources
- Reviews the core content from lectures 1-3
    - PPDAC
    - Variable types
    - Relationship between 2 variables
    - Intro to R
    - Data visualization

Key Technologies & Resources

bCourses:

Week 1: Introduction and working with data

Learning Objectives   Online Study Materials   Class Time   Assignments   Additional Resources

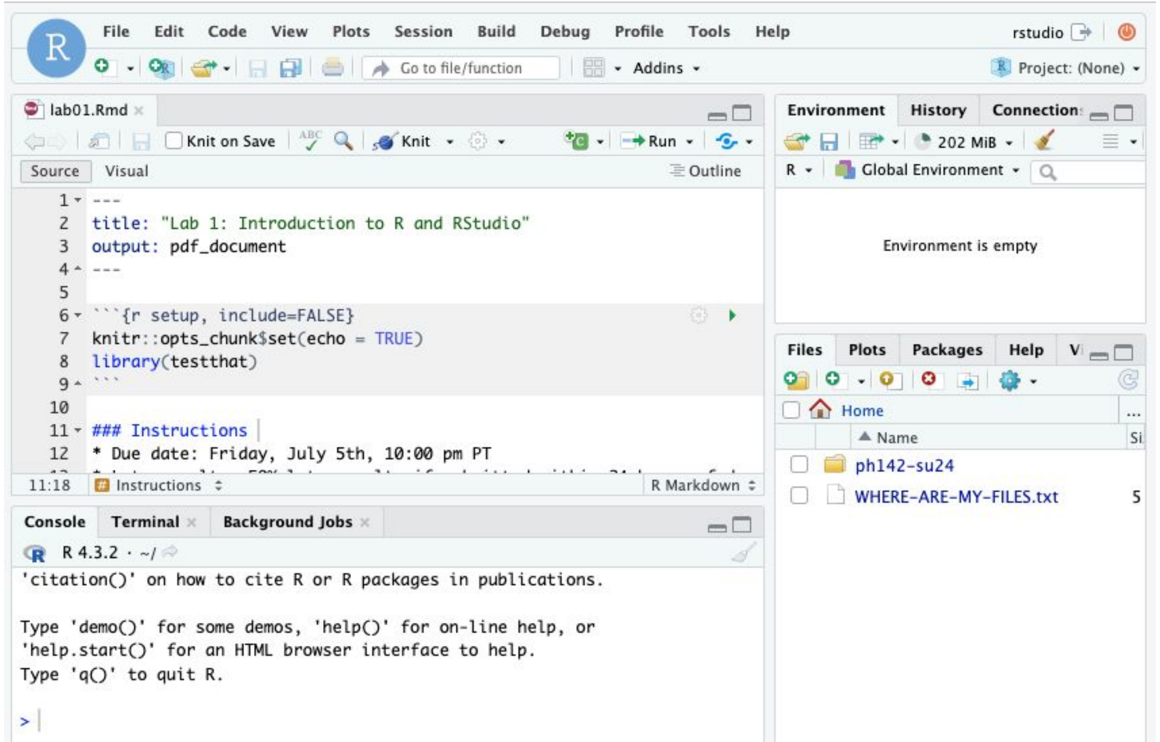
Learning Objectives

This week, you will learn to:

- Access R and R studio through the datahub
- Open and explore a dataset
- Create some visual and numeric summaries
- Estimate a line of best fit using linear regression

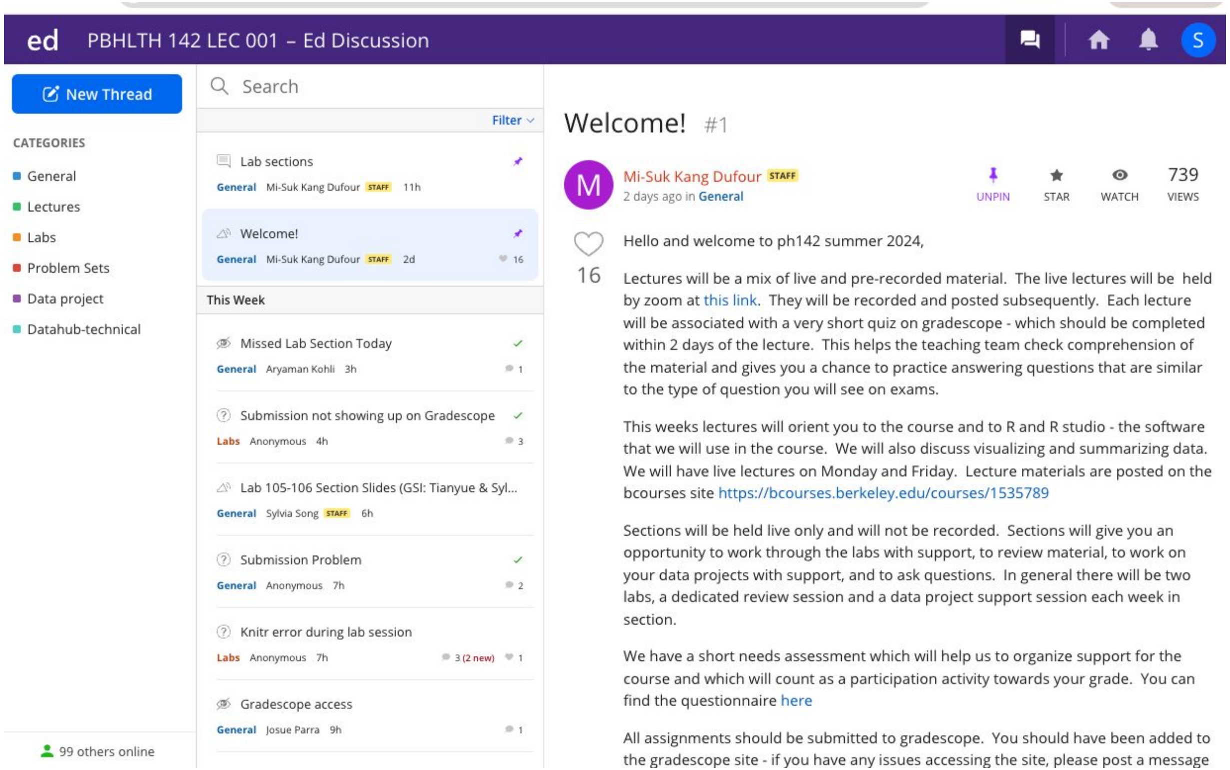
Key Technologies & Resources

DataHub:



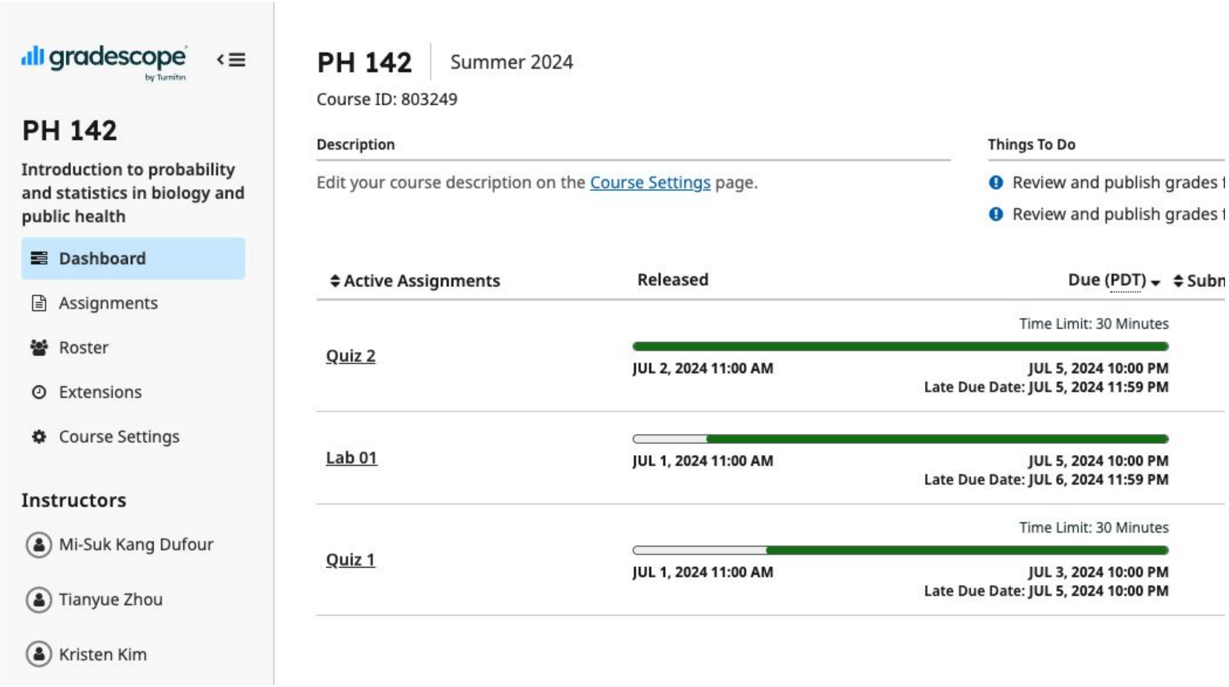
Key Technologies & Resources

Ed:



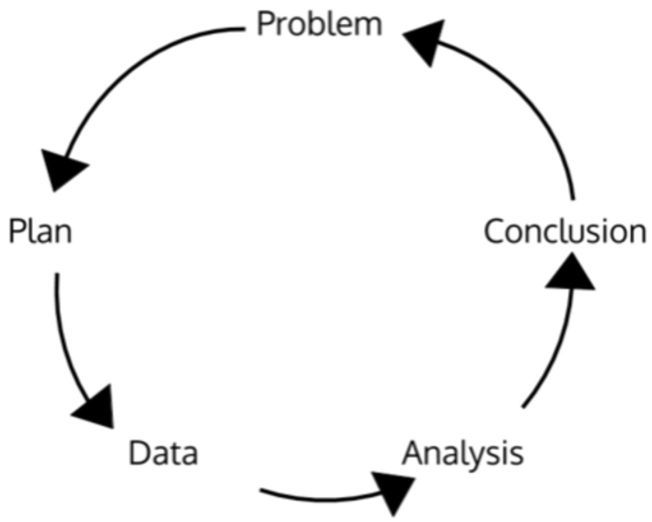
Key Technologies & Resources

Gradescope:



PPDAC Approach

- **Problem:** A clear statement of what we are trying to achieve
  - problem type – descriptive, predictive, causative/etiologic
- **Plan**: the procedure we use to carry out the study
- **Data**: data collect based on the plan
- **Analysis**: summarization and analysis of the data to answer questions posed by your problem
- **Conclusion**: what you learned from your answers to the problem



Types of Variables

- **Categorical**: a variable that has grouping levels
  - **Nominal**: no underlying order or rank, e.g. blood type, zip code
  - **Ordinal**: with an underlying order or rank, e.g. blood pressure level (low, normal, high)
- **Quantitative**: a numeric variable which you can perform mathematical operations on
  - **Discrete**: can be counted, e.g. the number of cookies in the bag you got from a bakery
  - **Continuous**: can be measured precisely, with a rule or scale, e.g. 5.34 grams of cornstarch

Relationship between 2 variables

Two variables

- X: explanatory variable, independent variable
- Y: response variable, dependent variable
- Scatter plot is helpful to visualize their relationship, and we should pay attention to:
    - The overall pattern
    - Form
    - Strength
    - Any exceptions to the overall pattern
    - Any outliers

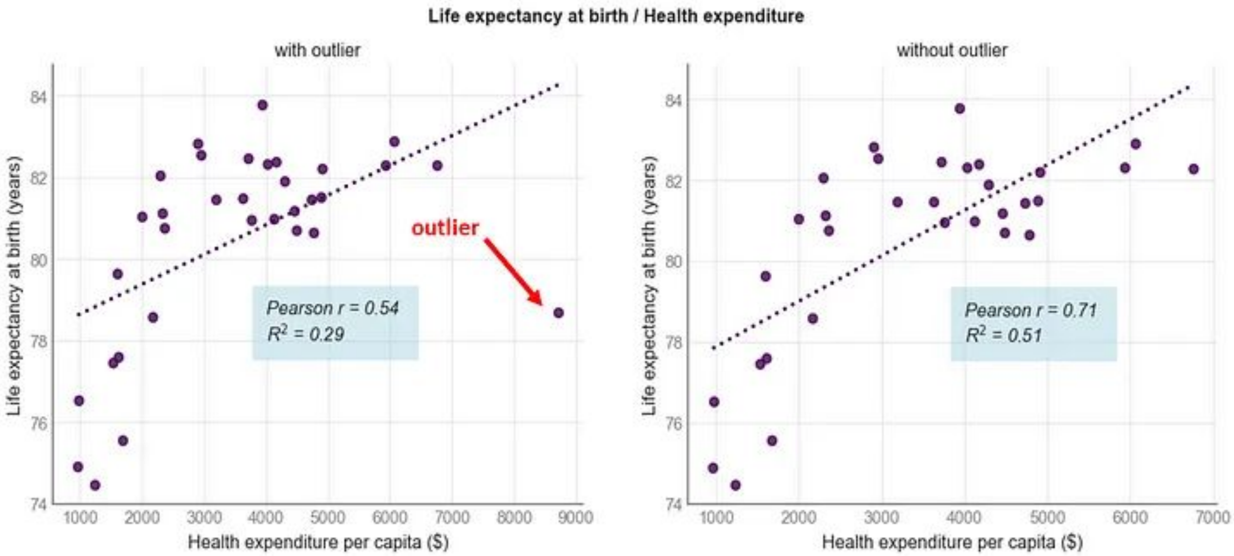
Relationship between 2 variables

Pearson's correlation

- -1 to 1
- 0 indicates no association
- Only useful when the relationship is linear

$$r = \frac{1}{n-1}\sum_{i=1}^{n}\left(\frac{x_i - \bar{x}}{s_x}\right)\left(\frac{y_i - \bar{y}}{s_y}\right)$$

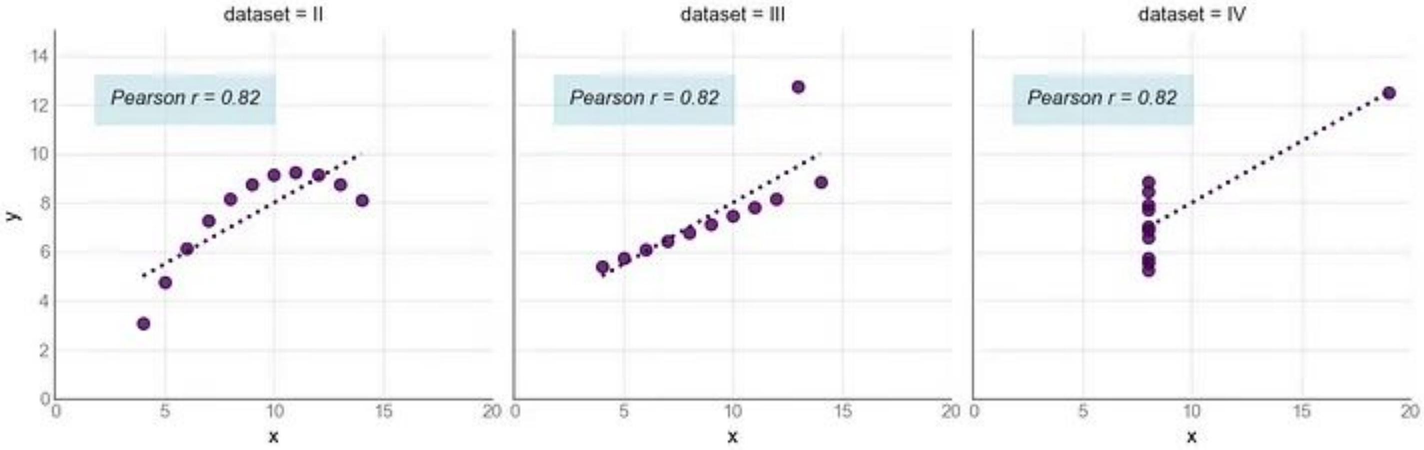
Relationship between 2 variables



Influence of an outlier

[Picture source]

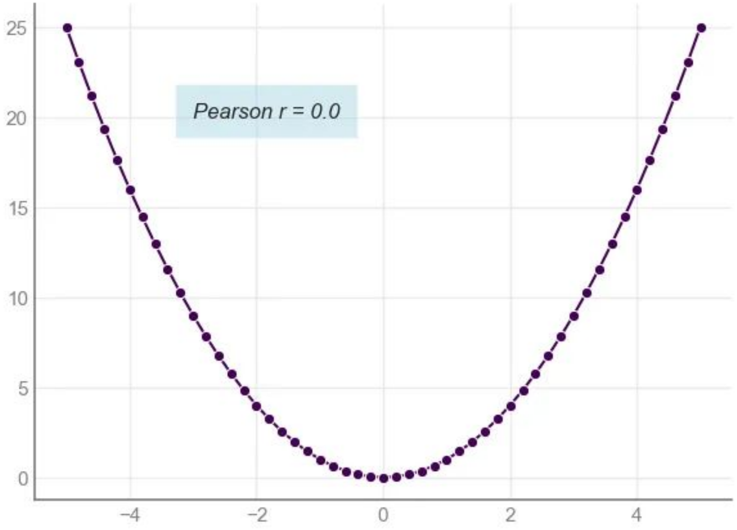
Relationship between 2 variables



High correlation doesn't indicate a linear relationship

Picture source

Relationship between 2 variables



No correlation doesn't mean no relationship

Picture source

Intro to R

- Library
  - A library is a package of functions, and you can load this package of functions by running library(ggplot2), library(dplyr)
  - Make sure you have them first, otherwise you need to do install.packages() first
- Read your data: e.g. how to read a .csv file
  - `library(readr)`
  - `mydata <- read_csv('my_data.csv')`
- Some functions to get a quick look of your data
  - head(mydata): shows the first 6 rows of the dataset
  - dim(mydata): shows the total number of rows by the total number of columns
  - names(mydata) or colnames(mydata): shows all the variable names (column names) of the dataset
  - str(mydata): summarizes the information above and more

Data manipulation: functions in library(dplyr)

First, do `library(dplyr)` to have the package in your environment.

- rename() → renames variables (columns)
  - `new_dataset <- old_dataset %>% rename(new_name = old_name)`
  - or: `new_dataset <- rename(old_dataset, new_name = old_name)`
- select() → subsets variables (columns)
  - `smaller_data <- old_data %>% select(variable1, variable2, variable3)`
  - `smaller_data <- select(old_data, variable1, variable2, variable3)`
  - `smaller_data <- select(old_data, variable1:variable3)`
  - To keep all variables other than variable1: `smaller_data <- old_data %>% select(- variable1)`
- arrange() → orders observations (rows) by a certain variable (column) or variables (columns)
  - `lake_data %>% arrange(ph)`
  - `lake_data %>% arrange(age_data, ph)`

Data manipulation: functions in library(dplyr)

- filter() → selects a subset of rows by certain conditions
  - If we want condition A AND condition B to be satisfied, use **,** or **&**
  - If we want condition A OR condition B to be satisfied, use **|** or **%in%**
    - `lake_data %>% filter(age_data == "recent")`
    - `lake_data %>% filter(lakes %in% c("Alligator", "Blue Cypress"))`
    - `lake_data %>% filter(ph > 6 | chlorophyll > 30)`
- mutate() → creates new variables
    - `lake_data_new <- lake_data %>% mutate(actual_fish_sample = number_fish_sampled * 100)`
- group_by() → groups the data by a categorical variable
    - `lake_data %>% group_by(age_data)%>% summarize(mean_ph = mean(ph))`
- summarize() → applies summary functions to calculate statistics
    - `lake_data %>% summarize(mean_ph = mean(ph), sd_ph = sd(ph))`

Visualization of Categorical Data: “ggplot2”

1. Install and load the “ggplot2” package
    a. install.packages(“ggplot2”)
    b. library(ggplot2)
2. Specify your data, and what to have on the x and y axes
3. Create a plot: geom_ functions (many options)
    a. geom_bar, geom_histogram, geom_point, geom_line
    b. Tip: picture how you want to visualize your data in your head first, then pick the function that helps you achieve your goal :)
4. Change the style of your plot
    a. labs() function: update your main title, axis names, caption
    b. theme() function: change the size of your title, font, and position

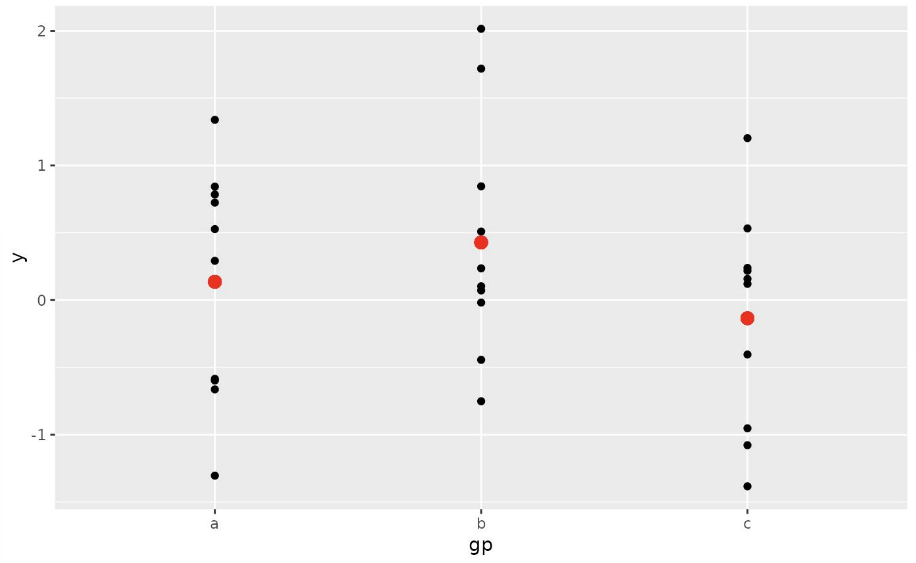
```
ggplot(data = opi_data, aes(x = Mean)) +
geom_histogram(col = “white”) +
labs(x = “Mean opioid prescription rate (per 100 individuals)”,
   y = "Number of states") +
theme_minimal(base_size = 15)
```

Visualization of Categorical Data: geom_pont()

To learn more:

- https://ggplot2.tidyverse.org/
- *R for Data Science*: https://r4ds.hadley.nz/data-visualize

```
ggplot(df, aes(gp, y)) +
  geom_point() +
  geom_point(data = ds, aes(y = mean), colour = 'red', size = 3)
```



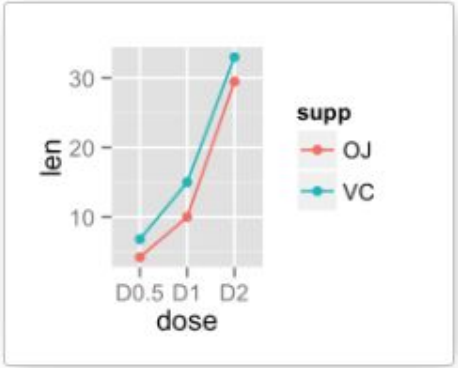
source: https://ggplot2.tidyverse.org/reference/ggplot.html

Visualization of Categorical Data: geom_line()

To learn more:

- https://ggplot2.tidyverse.org/
- *R for Data Science*: https://r4ds.hadley.nz/data-visualize

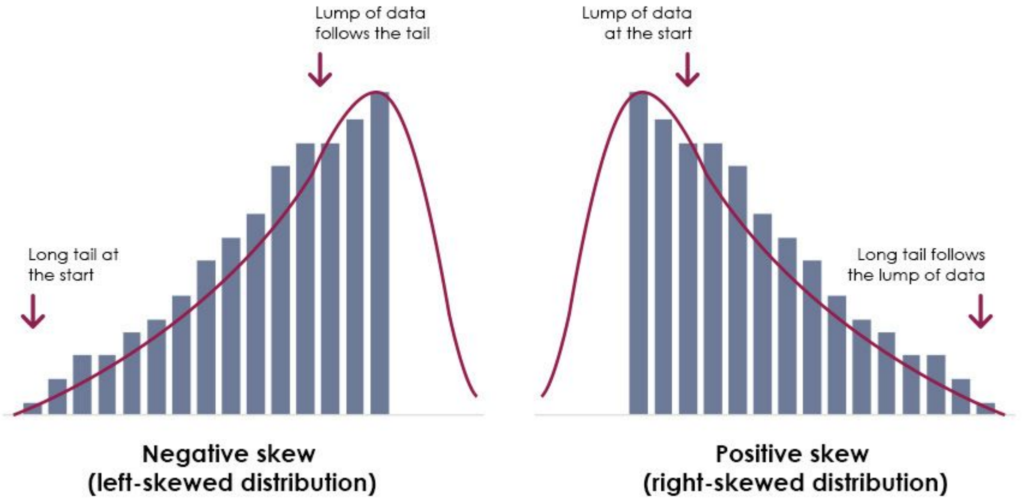
```
p<-ggplot(df2, aes(x=dose, y=len, group=supp)) +
  geom_line(aes(color=supp))+
  geom_point(aes(color=supp))
p
```



Picture source:
http://www.sthda.com/english/wiki/ggplot2-line-plot-quick-start-guide-r-software-and-data-visualization

Measure of Central Tendency

- Mean and median are **approximately equal** when...
  - Distribution is symmetric
  - Data has one peak
  - There are no outliers
- Outliers: large effect on the mean
- Skewed data: mean ≠ median
  - Skewed right: mean > median
  - Skewed left: mean < median



Picture source: https://www.cambridgemaths.org/blogs/skewed-usage-skewed-distribution/

Measures of Spread

- Range = max - min
- IQR = Q3 - Q1
    - Five number summary in R!

```
CS_dat %>% summarize(min = min(cs_rate),
Q1 = quantile(cs_rate, 0.25), median = median(cs_rate),
Q3 = quantile(cs_rate, 0.75), max = max(cs_rate))
```

- Sample variance (s^2)
- Sample standard deviation (s)

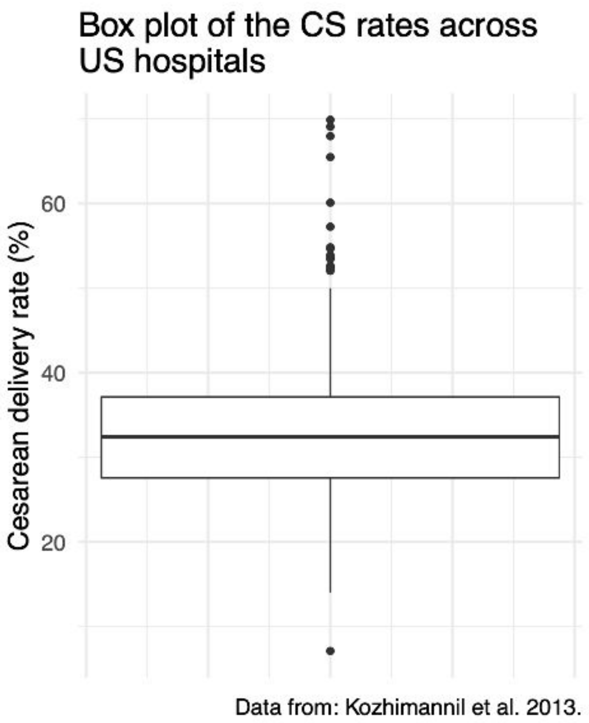
```
CS_dat %>% summarize(cs_sd = sd(cs_rate), cs_var = var(cs_rate))
```

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Box plot

- Center line → median
- Top of box → Q3
- Bottom of box → Q1
- Top of top whisker → max value or highest point that is below Q3 + 1.5*IQR
- Bottom of bottom whisker → min value or lowest point that is above Q1 - 1.5*IQR
- Data points above and below whiskers → outliers

```
ggplot(CS_dat, aes(y = cs_rate)) +
geom_boxplot() +
ylab("Cesarean delivery rate (%)") +
labs(title = "Box plot of the CS rates across US hospitals",
    caption = "Data from: Kozhimannil et al. 2013.") +
theme_minimal(base_size = 15) +
scale_x_continuous(labels = NULL) # removes the labels from the x axis
```



Common Errors

- Typo ;)
- Used a function from a package that wasn't loaded yet.
- Called a function on an object/variable that has the wrong type.
- Two different code chunks shared the same name.
- The same variable names that are listed in the instructions are used in your work.
- Data/code didn't run – try reloading your previous code chunks first.
- How to examine the output of an object or a line of code in rmd – simply run the code or the name of the object in the code chunk.

See you next week!